

EXPENSE TRACKER-PROJECT REPORT

Submitted by: - Gargi Namdev Registration no.: - 26BAI10950

Course: - Introduction to Problem Solving and Programming

Platform: - VITyarthi - Build Your Own Project

1. Introduction: -

Keeping track of daily spending is a routine but often neglected part of personal money management. Personal Finance & Expense Tracker is a desktop application designed to give users a quick, private way to record and monitor their everyday expenses. Unlike spreadsheets or cloud-based finance apps that require sign-ups and bank access, it stores all data locally on the user's own machine, allowing students, professionals, and general users to log expenses by category and instantly see how much they have spent, without any internet connection or third-party data sharing.

2. Problem Statement: -

The Problem:

Manual expense tracking involving notebooks or random receipts usually ends with some expenses not being tracked at all leading to a loss of insight into where the money has gone.

The current finance apps require creating an account and giving away personal banking information which is scary for people who just want to track some minor expenses.

The Solution:

A python based client side application which allows a user to input an expense along with a category and a note in a matter of clicks, validates the input and stores it in a persistent local storage in a form of a JSON file without the need of any external account or internet connection.

3. Functional Requirements:-

The system has three modules :

- 1) Expense Entry/Validation (Input Process)
- 2) Category Handling/View (User Interface)
- 3) History/Report (Storage Mechanism)

4. Non-Functional Requirements (Quality):-

These are attributes which define quality of a system. They are;

- 1) Performance: Adding expense, saving to data file and updating total should be done instantly without any delays.
- 2) Reliability (Offline by design): Since the application is intended to work offline, it should be reliable since it does the same thing every time. It should also open up with empty expense list if the data file is not found or empty instead of crashing.
- 3) Usability: The application should have graphical interface with clear texts (Times new roman font style) and labels. The user should log expense in less than three clicks.
- 4) Error handling: The software should not break down or terminate abruptly on invalid inputs for example; empty fields, use of letters and negative numbers. It should also provide warning messages and/or prompt to the user instead of terminating the process.

5. System Architecture: -

The project has a Modular Architecture, which breaks up the system into parts based on their features. These are the following layers:-

- Presentation Layer (GUI):

It has all the user interface and input functionalities. This is built with Tkinter, a Python package for developing GUI applications. The GUI displays the list of expenses and the running total of the expense.

- Logic Layer (Validation and Totals):

It holds the pure Python Functions which handle the input data validations and calculation of the running total of the expenses.

- Data Layer (Local Storage):

It is managed by Python functions `load _expenses()` and `save _expenses()`, which perform the loading and saving of all the expenses data to a file called `expenses.json`.

6. Design Diagrams

6.1 Use Case Diagram:-

Actors: Users Use cases:-

- Enter amount
- Select category
- Add a note (optional)
- Add expense
- View expense list
- Delete selected expenses
- View running total

6.2 Workflow Diagram

Launch the app → load the expenses list saved in JSON → user inputs the money amount, expense category, and note → check the inputs → save the new expense → update the list and total expenses → end.

6.3 Sequence Diagram

1. User inputs the money amount, selects the category, and clicks the Add Expense button;
2. GUI reads the value of the amount field and the note field content;
3. if the amount is empty, not numeric, or less than zero, show an error message;
4. otherwise, create a new expense object with the current date, selected category, amount, and note;
5. append the new expense to the expenses list;
6. `save_expenses()` saves the updated expenses list in `expenses.json` file;
7. GUI triggers `refresh_list()` and `update_total()` methods.

7 Design Decisions & Rationale: -

- Language: - Chose python due to ease of use & file handling, JSON storage is baked into the language
- GUI Framework: - Tkinter was the obvious choice seeing as the requirement was a lightweight standalone desktop application (as opposed to web app requiring a browser)
- Storage: - Local JSON file because it's a personal expense tracker, need not bother with a database or external API, the app should work offline with minimum requirements

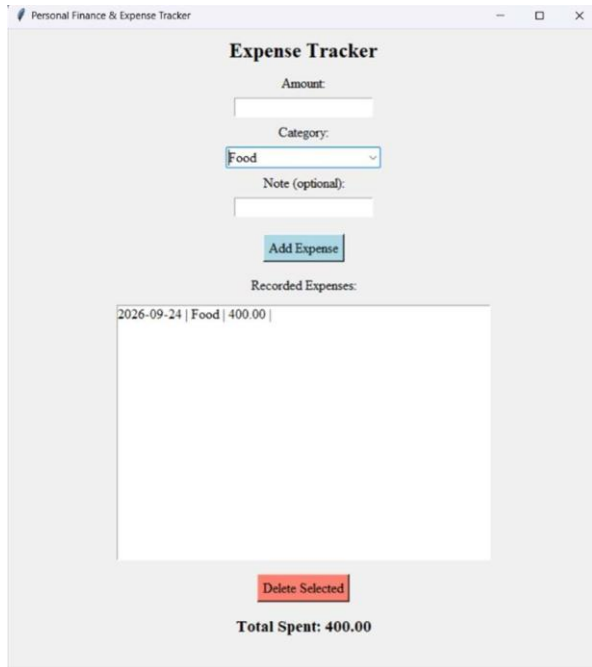
8 . Screenshots :-

1. Entering the amount, category, and note for a new expense

The screenshot shows a window titled "Personal Finance & Expense Tracker" with a subtitle "Expense Tracker". It contains the following elements:

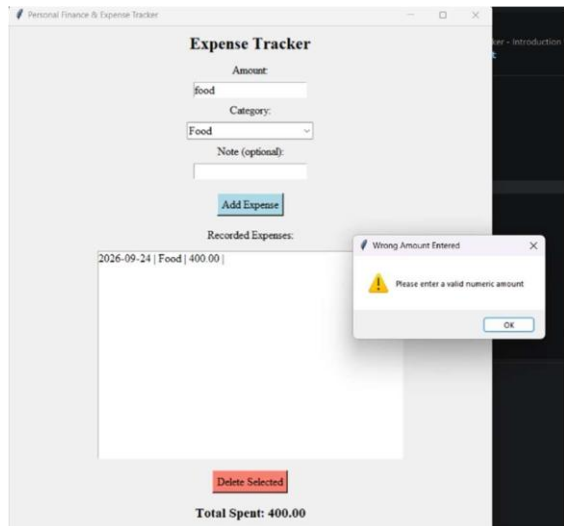
- An "Amount:" label above a text input field containing the value "400".
- A "Category:" label above a dropdown menu.
- A dropdown menu with the following options: Food (selected), Transport, Rent, Shopping, Entertainment, Bills, and Other.
- A "Recorded Expenses:" label above a large, empty rectangular box.
- A red button labeled "Delete Selected" below the recorded expenses box.
- A "Total Spent: 0.00" label at the bottom of the window.

2. Adding the expense in the record



The screenshot shows the 'Expense Tracker' application window. At the top, the title bar reads 'Personal Finance & Expense Tracker'. The main area is titled 'Expense Tracker'. It contains a form with the following fields: 'Amount:' (a text input field), 'Category:' (a dropdown menu with 'Food' selected), and 'Note (optional):' (a text input field). Below these fields is a blue 'Add Expense' button. Underneath the button, the text 'Recorded Expenses:' is followed by a list box containing the entry '2026-09-24 | Food | 400.00 |'. At the bottom of the window, there is a red 'Delete Selected' button and a label 'Total Spent: 400.00'.

3. Warning generated when no amount is entered and "Add Expense" is pressed



The screenshot shows the 'Expense Tracker' application window with an error message displayed. The form fields are the same as in the previous image, but the 'Amount:' field now contains the text 'food' instead of a numeric value. The 'Add Expense' button is highlighted. A modal dialog box titled 'Wrong Amount Entered' is open in the foreground. It contains a yellow warning icon and the text 'Please enter a valid numeric amount'. There is an 'OK' button at the bottom of the dialog. The background application window shows the same recorded expenses and total spent as before.

9. Testing Approach: -

the total-calculation logic was tested with known values (e.g., three expenses of 100 each should ensure accuracy).

ing: Tested inputs with 0, negative numbers, blank text, and non-numeric text to ensure the c holds.

sting: The application was closed and reopened after adding expenses to verify that all entries y from expenses.json.

10.Challenges Faced: -

- Ensuring the expense list and total stayed correctly in sync after every add or delete action.
- Handling a missing or empty expenses.json file gracefully on the very first run.
- Designing input validation that rejects blank, non-numeric, and zero/negative amounts without confusing the user.

11.Learnings & Key Takeaways: -

- Gained practical experience in reading and writing JSON data in Python.
- Learned the importance of Modular Design - separating data-handling functions from the GUI made the code easier to follow and debug.
- Understood how to manage Data Persistence using simple local file handling instead of a database.

12.Future Enhancements: -

- Expense Graphs: Add a visualization module (using Matplotlib) to show spending by category.
- Monthly Budget Limits: Allow users to set a monthly budget and warn them when they are close to exceeding it.
- Export to CSV/Excel: Let users export their expense history for use in other tools.

13.References

1. Python Documentation: <https://docs.python.org/3/>
2. Tkinter Reference: <https://tkdocs.com/>
3. Python JSON Module Documentation: <https://docs.python.org/3/library/json.html>